

Library & Resource Management System

Object-Oriented Programming — CS-116 Team Project Report

FE Batch 2025 | Spring Semester 2026

Field	Details
Group ID	G4-9
Section	C
Course	Object-Oriented Programming (CS-116)
Instructor	Miss Fauzia Yasir

Group Members

Name	Roll Number
Muhammad Hamza Ahsan	CS-130
Shahzad Ahmed Siddiqui	CS-127
Muhammad Haseeb	CS-120
Muhammad Moosa Iqbal	CS-143

TABLE OF CONTENTS

TABLE OF CONTENTS	2
1. Problem Description	2
2. Distinguishing Features & OOP Concepts Implemented	3
2.1 Inheritance	4
2.2 Abstraction	4
2.3 Polymorphism — Runtime Dispatch	4
2.4 Encapsulation	4
2.5 Composition and Aggregation	4
2.6 Exception Handling	5
2.7 File Handling — Flat-file Persistence	5
3. Flow of the Project — Class Diagram	5
4. Most Challenging Part	6
5. New Things Learned in C++	7
6. Individual Contributions	8
Muhammad Hamza Ahsan (CS-130)	9
Shahzad Ahmed Siddiqui (CS-127)	9
Muhammad Haseeb (CS-120)	9
Muhammad Moosa Iqbal (CS-143)	9
7. Future Expansions	9
8. Test Case Runs	10
TEST CASE 1:	11
TEST CASE 2:	11
TEST CASE 3:	12

1. Problem Description

The Library & Resource Management System was developed to address the everyday operational challenges faced by modern libraries. Traditional libraries often struggle with manual record-keeping, inefficient resource tracking, and the lack of automated fine management — all of which lead to errors, delays, and poor user experience.

This project provides a console-based C++ application that automates core library operations. The system supports five distinct user roles — Student, Teacher, Staff, Premium Member, and Admin — each with its own borrowing rules, fine rates, and privileges. It also manages five types of library resources: Books, DVDs, AudioBooks, Magazines, and Newspapers.

The system handles the following core operations:

- Registration and role-based login for all user types
- Borrowing and returning resources with daily limits enforced per user role
- Automatic overdue fine calculation deducted from user balance
- Full borrow history tracking with dates and fine records
- Admin capabilities: add, update, soft-delete resources and users, view reports, export data
- Data persistence across sessions using CSV flat-file storage
- Automatic membership upgrade when a user's balance reaches a defined threshold

All data is stored in CSV files located in the database/ folder. On startup, these files are read into memory. On clean exit, all changes are saved back. This ensures the system retains its full state between sessions without requiring a database engine.

2. Distinguishing Features & OOP Concepts Implemented

This project goes beyond the minimum requirements by incorporating all major Object-Oriented Programming principles in a cohesive, working system. Below is a description of each OOP concept and where it appears in the codebase.

2.1 Inheritance

Inheritance is used through two main hierarchies: User is inherited by Student, Teacher, Staff, PremiumMember, and Admin, while LibraryResource is inherited by Book, DVD, AudioBook, Magazine, and Newspaper. The base classes provide common attributes and structure, while each derived class adds its own specific data and behavior. Constructors of derived classes call the base class constructor using initializer lists and then initialize their own attributes. This approach avoids duplication by keeping shared logic in the base classes and only extending what is needed in child classes.

2.2 Abstraction

Abstraction is implemented by making both User and LibraryResource abstract classes using pure virtual functions. User class defines pure virtual methods for daily limit, fine rate, borrow days, and display information, while LibraryResource defines a pure virtual display method. These base classes are never instantiated directly; instead, only their derived classes are used. The Library class stores all users and resources using base-class pointers and interacts with them through these abstract interfaces, allowing different derived objects to be handled uniformly while still calling their correct implementations at runtime.

2.3 Polymorphism — Runtime Dispatch

Polymorphism is primarily implemented by storing diverse derived objects into a vector of base-class pointers. This allows the File Handler to dynamically instantiate specific types based on CSV data and assign them to a universal base pointer. Furthermore, the system uses virtual functions to automatically invoke the correct logic for different functions depending on the object's actual type.

2.4 Encapsulation

All data members are private or protected. Public access is only through typed getters and setters defined in each class. No field is exposed directly.

2.5 Composition and Aggregation

Composition

Composition is used between Library and BorrowRecord. BorrowRecord objects are stored inside the borrowHistory vector of the Library class and only exist as part of the Library system. If the Library is removed, its borrow records are also removed.

Aggregation.

Aggregation is used between Library and both User and LibraryResource. The Library class stores User and LibraryResource objects as pointers inside separate vectors to manage all users and resources in one place. However, these are independent classes with their own attributes and behaviors, so the relationship is aggregation.

Association

Association is used between BorrowRecord and LibraryResource. Each BorrowRecord stores a pointer to a LibraryResource to keep track of which item was borrowed. BorrowRecord only references the resource and does not control its lifecycle.

2.6 Exception Handling

Exception handling is integrated across all modules to prevent crashes from invalid input and file errors. In Utils.cpp, functions like `getValidInt()`, `getValidDouble()`, and `getValidString()` throw `invalid_argument` when parsing fails, while Library.cpp and Admin.cpp raise `runtime_error` for invalid user or resource types. FileHandler.cpp throws errors if CSV files are missing or inaccessible, and main.cpp wraps file operations in try/catch to continue safely. The Menu module also catches unexpected exceptions from any operation, ensuring the session remains stable and user-friendly.

2.7 File Handling — Flat-file Persistence

File Handling comprises a collection of standalone functions that manage the application's data persistence. Its primary responsibility is to act as a bridge between the physical files and the program's memory OR vectors, reading CSV rows to reconstruct objects and writing the current state back to files. Additionally, it maintains data integrity by scanning files to generate unique IDs for every new user or resource registration.

4. Most Challenging Part

Each team member encountered a distinct technical challenge during development. The table below summarizes these experiences.

Member	Challenging Experience
Muhammad Hamza Ahsan (CS-130)	Working on file handling while using aliases (references). Passing large vectors by reference to FileHandler functions — rather than by value — to avoid unnecessary copying and keep the in-memory data synchronized with disk storage.
Shahzad Ahmed Siddiqui (CS-127)	Implementing the borrowing and returning logic correctly. Ensuring that daily borrow limits were enforced per user, overdue fines were calculated accurately, and balance deductions happened atomically inside markReturned() without corrupting state.
Muhammad Haseeb (CS-120)	Working with vectors and accessing files using references. Managing dynamic collections of base-class pointers and ensuring that Library class functions correctly read from and wrote to these vectors without dangling pointers or data loss.
Muhammad Moosa Iqbal (CS-143)	Understanding and applying polymorphism in practice — specifically why calling getFineRate() on a User* returns different values for different derived types, and how dynamic_cast enables safe downcasting in the admin resource-update flow.

5. New Things Learned in C++

Each member acquired at least one new technical skill during development that was not part of their prior experience.

Member	New Skill / Learning
Shahzad (CS-127)	I learned to use the C++ <code><ctime></code> library (<code>time(0)</code> and <code>difftime</code>) to calculate overdue days for borrowed resources. I also gained practical experience with <code>std::vector</code> for dynamic storage and safe management of user and resource pointers.
Hamza (CS-130)	I gained practical experience organizing a large C++ project with clear separation of headers and source files, using include guards and proper compilation commands. I also learned to use references as aliases to avoid unnecessary copying, especially when passing large containers like vectors.
Haseeb (CS-120)	I acquired practical skills in C++ file handling using <code>fstream</code> , applying <code>getline</code> with delimiters and <code>stringstream</code> to parse CSV data. I also learned to manage persistent state by rebuilding objects from flat files on startup and serializing them back on exit.
Moosa Iqbal (CS-143)	I gained practical understanding of polymorphism by using vectors of base-class pointers, where virtual functions resolved correctly at runtime (e.g., <code>getFineRate()</code> returning different values for <code>Student</code> vs. <code>PremiumMember</code>). I also applied <code>dynamic_cast</code> in <code>Admin::updateResource</code> to safely downcast resources and access type-specific setters, demonstrating runtime type information in action.

6. Individual Contributions

All four team members participated across the full development lifecycle which includes system design, implementation, testing, and documentation with primary responsibilities as follows.

Muhammad Hamza Ahsan (CS-130)

Responsible for the foundational architecture of the project. His contributions include designing and implementing the database structure and CSV-based file handling system, organizing the codebase into a modular folder structure with clearly separated header and source files, and authoring the UML class diagram. He performed systematic debugging across the project, conducted regular code reviews of teammates' work throughout the development cycle, and implemented User and Resource class hierarchies, including associated header and source files. He also developed the application's title and startup screen, as well as the user membership upgrade feature. In addition, he provided support in the preparation of the project documentation and final report.

Shahzad Ahmed Siddiqui (CS-127)

Led the implementation of exception handling across the entire codebase, ensuring system robustness and fault tolerance. He was the primary developer for the admin module, delivering key functionalities such as resource management, user management, and reporting features. He also contributed to the architectural design of the UML class diagram and participated in structuring the main application entry point. Furthermore, he assisted the team in end-to-end debugging the application. In addition, he partnered with the team to put together the project documents and final report.

Muhammad Haseeb (CS-120)

Implemented several core Library class functions and cross-checked the desired output by manually executing and verifying each feature. He also contributed to a few admin functions and helped ensure the correctness of the overall system behavior. He also worked on adding eye-catching colors to the project. In addition, he facilitated the compilation of project documentation and the final report.

Muhammad Moosa Iqbal (CS-143)

Contributed to the implementation of few Library class functions and assisted in developing the User and Resource class hierarchies. He further assisted in the development of the title page and CLI's visual presentation by integrating a color-coded interface. In addition, he supported the creation of project documentation and the final report.

7. Future Expansions

The following improvements are planned for future versions of this system.

- Replace CSV flat-files with an SQL database (e.g., SQLite) for better performance, concurrent access, and proper relational integrity.
- Develop a web-based interface to replace the console UI, enabling access from any browser with a proper front-end design.
- Add an E-books portal — a dedicated section for digital resources with separate borrowing rules and a download/streaming interface.
- Implement an automated overdue notification system that sends messages (email or SMS) to users with overdue resources.
- Add a resource reservation system so users can hold an unavailable resource and be notified when it becomes available.
- Introduce damage fee deduction: if a resource is returned with damage, a configurable damage fee is deducted in addition to any overdue fine.
- Support multi-user concurrent access with a network layer, allowing multiple terminals or clients to connect to a shared library server.

8. Test Case Runs

Test Case 1:

--- Borrow History ---					
User ID	Resource	Borrowed	Due	Returned	Fine
469	Discrete Mathematics Vol 5	06-04-2025	13-04-2025	Not yet	3920
304	Daily Jang	09-04-2025	16-04-2025	17-04-2025	10
606	Software Engineering Vol 5	13-04-2025	13-05-2025	17-04-2025	0
598	Software Engineering Vol 5	14-04-2025	14-05-2025	Not yet	3610
141	Digital Logic	15-04-2025	22-04-2025	29-04-2025	70
114	Data Structures Vol 10	16-04-2025	23-04-2025	24-04-2025	10
476	Discrete Mathematics Vol 1	18-04-2025	25-04-2025	27-04-2025	10
377	Machine Learning	21-04-2025	28-04-2025	Not yet	3770
433	Software Engineering Vol 10	23-04-2025	30-04-2025	01-05-2025	0
216	Pakistan Today	25-04-2025	02-05-2025	03-05-2025	0
202	Titanic	28-04-2025	05-05-2025	12-05-2025	70
363	Operating Systems Vol 4	30-04-2025	07-05-2025	06-05-2025	0
373	Daily Jang	02-05-2025	09-05-2025	16-05-2025	70
139	Computer Networks Vol 6	05-05-2025	12-05-2025	10-05-2025	0
500	Operating Systems Vol 8	07-05-2025	14-05-2025	15-05-2025	0
15	Machine Learning Vol 4	09-05-2025	16-05-2025	23-05-2025	70
530	Operating Systems Vol 8	12-05-2025	26-05-2025	Not yet	6980
245	Operating Systems Vol 6	14-05-2025	21-05-2025	21-05-2025	0
463	Gladiator	16-05-2025	23-05-2025	23-05-2025	0
594	Computer Networks	19-05-2025	18-06-2025	Not yet	3260
180	Linear Algebra	21-05-2025	28-05-2025	29-05-2025	0
377	Daily Jang	23-05-2025	30-05-2025	06-06-2025	70
156	Scientific American	26-05-2025	02-06-2025	01-06-2025	0
253	The Dark Knight	28-05-2025	04-06-2025	11-06-2025	70
175	Scientific American	30-05-2025	06-06-2025	Not yet	3380
170	Tech Today	01-06-2025	08-06-2025	14-06-2025	50
170	Time	04-06-2025	11-06-2025	11-06-2025	0
464	Data Structures Vol 5	06-06-2025	13-06-2025	19-06-2025	50
298	Tech Today	08-06-2025	15-06-2025	22-06-2025	70
374	Interstellar	11-06-2025	18-06-2025	18-06-2025	0
269	Software Engineering Vol 10	13-06-2025	20-06-2025	Not yet	3240
414	Data Structures Vol 10	15-06-2025	22-06-2025	22-06-2025	0
470	Tech Today	18-06-2025	25-06-2025	Not yet	3190
262	Machine Learning Vol 8	20-06-2025	27-06-2025	27-06-2025	0
165	Linear Algebra Vol 6	22-06-2025	29-06-2025	05-07-2025	50
102	Database Systems	25-06-2025	02-07-2025	09-07-2025	70
206	Software Engineering Vol 1	27-06-2025	04-07-2025	Not yet	3100
320	Pakistan Today	29-06-2025	06-07-2025	06-07-2025	0

Test Case 2:**--- Overdue Resources ---**

User ID	Resource	Days Overdue
469	Discrete Mathematics Vol 5	392
598	Software Engineering Vol 5	361
377	Machine Learning	377
530	Operating Systems Vol 8	349
594	Computer Networks	326
175	Scientific American	338
269	Software Engineering Vol 10	324
470	Tech Today	319
206	Software Engineering Vol 1	310
307	Operating Systems Vol 1	301
561	Computer Networks Vol 8	291
78	Software Engineering Vol 1	294

Press Enter to return to menu...█

Test Case 3:

```

--- Fine Management (Overdue Report) ---

```

User ID	Name	Resource	Days Overdue	Fine (Pending)
469	Imran Sheikh	Discrete Mathematics Vol 5	392	3920
598	Noman Shah	Software Engineering Vol 5	361	3610
377	Danish Farooq	Machine Learning	377	3770
530	Bilal Khan	Operating Systems Vol 8	349	6980
594	Hussain Rehman	Computer Networks	326	3260
175	Hassan Rehman	Scientific American	338	3380
269	Hussain Hussain	Software Engineering Vol 10	324	3240
470	Farhan Ansari	Tech Today	319	3190
206	Usman Qureshi	Software Engineering Vol 1	310	3100
307	Kamran Baig	Operating Systems Vol 1	301	3010
561	Bilal Farooq	Computer Networks Vol 8	291	5820
78	Noman Baig	Software Engineering Vol 1	294	2940

Press Enter to return to menu...

The following test scenarios were executed and verified:

- **Test Case 1:** Borrow History Tracking: A student borrows multiple resources over time, including books, magazines, and journals. The system correctly records borrow dates, due dates, return status, and pending fines in the borrow history report.
- **Test Case 2:** Overdue Resource Detection: The system scans all borrowed resources and identifies overdue items by comparing the current date with the due date. It correctly displays overdue resources along with the number of overdue days for each user.
- **Test Case 3:** Fine Management Report: Admin accesses the fine management module to review users with pending overdue fines. The system accurately calculates fines based on overdue days, displays user details with borrowed resources, and generates a complete overdue fine report successfully.